NeuroVista — Technical Stack Reference

Detailed reference for technical Q&A. Covers hardware, on-device models, custom pipeline nodes, biomarker math, persistence, dashboard, voice, and the design decisions behind each choice.

1. Hardware

COMPONENT	SPEC	WHY IT MATTERS
Luxonis OAK 4 D R9	RVC4 SoC, 4× IMX378 sensors (1× RGB + 1× left mono + 1× right mono + 1× wide), DSP for NN inference, integrated stereo block	RVC4 runs the full vision pipeline on-device; CPU host (Mac) only post-processes ~200 B/frame metric payload
Luxonis OS	RVC4 1.32.0 (we ran on this; 1.20.5 had a static-IP bug we avoided by using DHCP link-local)	Provides DeviceGate session API, OAK App container runtime, oakctl CLI
PoE+ injector	30 W (NOT plain PoE, NOT USB)	The OAK 4 D draws ~6–8 W under load; plain PoE (15 W) browns out; USB cannot deliver the wattage
Network	Direct Ethernet, link-local IPv4 (169.254.x.x via auto-IP), no router required	Simplest setup; sub-ms latency Mac↔OAK
Connection	`oakctl device connect <ip>`; OAK Viewer also available for manual checks at `/Applications/OAK Viewer.app`	OAK Viewer must be **disconnected** before our pipeline runs (only one process can grab the device gate)

OS / runtime: - Host: macOS 14 (Apple Silicon), Python **3.9.6** (system) — chosen because `depthai 3.6.1` and `depthai-nodes 0.3.6` ship wheels for it. Homebrew Python 3.14 is **incompatible** (no depthai wheel).

2. On-device pipeline (RVC4 DSP)

ASCII diagram of the full on-device graph (every box runs on the OAK, **not** on the Mac):

```
NEUROVISTA · LONGITUDINAL SCREENING ·    GDG AI HACK 2026 · TRACK B ·
ON-DEVICE · PRIVACY-FIRST                LUXONIS OAK 4 D
```

```
┌──────────────────────────────────────────────────────────────────────────────┐
│  OAK 4 D — RVC4 SoC                                                          │
│                                                                              │
│  CAM_A (RGB)        CAM_B (mono left)        CAM_C (mono right)              │
│  IMX378 1024×768    IMX378 640×400           IMX378 640×400                  │
│       │                  │                        │                          │
│       │                  └────────────────────────┘                          │
│       │                              ▼                                       │
│       │                ┌────────────────────┐                                │
│       │                │ StereoDepth        │  preset: HIGH_DETAIL           │
│       │                │ left/right rectified │  setLeftRightCheck=True      │
│       │                │ depth aligned to A  │  setDepthAlign(CAM_A)        │
│       │                └────────────────────┘                                │
│       │                            │ depth uint16 mm 640×400                 │
│       │                            │                                         │
│       │ ImageManip                 │                                         │
│       │ resize to 320×240          │                                         │
│       ▼                            │                                         │
│  ┌──────────┐                      │                                         │
│  │  YuNet   │  face detection      │                                         │
│  │ (ParsingNN)│ INT8 RVC4           │                                         │
│  │ 320×240  │  ~600 inf/s          │                                         │
│  └──────────┘                      │                                         │
│       │ ImgDetectionsExtended (bbox + 5 kp + score)                          │
│       │                            │                                         │
│       ▼                            │                                         │
│  ImgDetectionsBridge (compat shim) │                                         │
│       │                            │                                         │
│       ▼                            │                                         │
│  Script node: crop face region     │                                         │
│  to 192×192 using bbox             │                                         │
│       │                            │                                         │
│       ▼                            │                                         │
│  ┌──────────────────┐              │                                         │
│  │ MediaPipe Face   │              │                                         │
│  │ Landmarker       │              │                                         │
│  │ (ParsingNN)      │              │                                         │
│  │ 192×192 → 1404 fl.│             │                                         │
│  │ INT8 RVC4        │              │                                         │
│  └──────────────────┘              │                                         │
│       │ Keypoints (468 × x,y,z)    │                                         │
```

```
│  GatherData (sync detection ⊕ landmarks ⊕ depth)                    │
│       │                                                              │
└───────┼──────────────────────────────────────────────────────────────┘
        │ (single sync'd packet sent to host every ~40 ms)
        ▼
   ── HOST (Mac) — Python ──
   BiomarkerExtractor → OverlayNode → VisitWriter → Visualizer (WebRTC)
```

Power budget (measured / specced): - StereoDepth + 2 NN @ 25 FPS sustained: **~2.5–3 W** on the DSP - Total device draw under load: **~6–8 W** (dominated by image sensors)

3. Models

3.1 YuNet — face detection

FIELD	VALUE
Source	Wu et al., "YuNet: A Tiny Millisecond-level Face Detector", Shanghai Jiao Tong / OpenCV maintainers, 2018; in OpenCV 4.5.4+ as `cv::FaceDetectorYN`
Manifest	`depthai_models/yunet.RVC4.yaml` → `model: luxonis/yunet:320x240, platform: RVC4`
Input shape	`[1, 3, 240, 320]` BGR (NCHW)
Output	bbox + score + 5 landmark per face (left/right eye, nose tip, mouth corners)
Architecture	Encoder-decoder. **Downconv path** = depthwise-separable Conv 3×3 stride 2 (MobileNet-style). **Upconv path** = bilinear upsample 2× + 1×1 conv lateral, FPN-like. **Multi-scale anchor-free heads** (one per FPN level).
Total params	**~85 000**
Quantization	INT8, custom calibration on RVC4
Throughput	**~600 inferences/s** on RVC4 DSP (Luxonis spec)
Why not MTCNN/Haar/RetinaFace	MTCNN: too slow (~50 ms), three-stage cascading; Haar: not robust to yaw/lighting; RetinaFace: 23M params, overkill on OAK; YuNet: 85k params, anchor-free, native OpenCV
Where used in code	`main.py:55–58` (loads from Zoo) → `main.py:104–106` (`ParsingNeuralNetwork.build`)

Caveat: YuNet bbox loses confidence beyond ~30° head yaw. We mitigate client-side with the head-pose gate (see §6.4).

3.2 MediaPipe Face Landmarker — facial mesh

FIELD	VALUE
Source	Bazarevsky et al., "Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs", Google Research, CVPR Workshops 2019 (BlazeFaceMesh derivative)
Manifest	`depthai_models/mediapipe_face_landmarker.RVC4.yaml` → `model: luxonis/mediapipe-face-landmarker:192x192, platform: RVC4`
Input shape	`[1, 3, 192, 192]` BGR — receives the face crop produced by the Script node, not the full frame
Output shape	`[1, 1, 1, 1404]` (= 468 landmarks × 3 coords) + `[1, 1, 1, 1]` (face presence score)
Architecture	**Encoder-only CNN** (no upconv path needed: regression task, not dense prediction). Stack: 5 stages of BlazeBlock (depthwise + pointwise + residual) downconv, 192→96→48→24→12→6. Bottleneck 6×6×192 → flatten → FC 512 → FC 1404 → reshape (468, 3).
Total params	**~600 000** ("lite" variant — what we use)
Quantization	INT8
Throughput	~600 inf/s on RVC4
Coords	x, y in `[0, 1]` of the **face crop** (NOT of the original frame); z is unitless relative depth (head-pose proxy, not metric)
Note on rectification	Because output coords are crop-relative, we map them to full-frame coords using the YuNet bbox (`utils/biomarker_node.py:103–119`). Without this step, the smile/brow distance computation falls onto the wall behind the patient.

Why this model and not 478-pt iris-equipped variant: Luxonis Zoo only ships the 468 (no iris) build. Adding iris would require model conversion through `luxonis/modelconverter` (~2-3 h, risk of build failure). Decided not worth the risk during the 24 h hack window.

3.3 StereoDepth

FIELD	VALUE
Class	`dai.node.StereoDepth` (built into DepthAI core)
Inputs	Two rectified mono streams from CAM_B and CAM_C, `640×400` @ 30 FPS
Preset	`HIGH_DETAIL` (denser disparity map, ~50% more compute than DEFAULT)
`setLeftRightCheck(True)`	Filters occluded pixels (depth value 0 where disparity is inconsistent left↔right)
`setDepthAlign(CAM_A)`	Reprojects depth into the RGB sensor frame, so pixel `(u,v)` in RGB has the corresponding depth in the depth output at the same `(u,v)`. **Critical** for 3D landmark reconstruction
Output	`uint16` depth in mm, shape `(400, 640)`. We resample landmark pixel coords from RGB-space (1024×768) to depth-space (640×400) before lookup (`utils/biomarker_node.py:178-182`)
Median filtering	Built-in 5×5 median, robust to single-pixel outliers
Sanity-clip in our code	`face_distance > 1500 mm` → set to `None` (FaceMesh can hallucinate landmarks when face leaves the crop, depth then reads the back wall — see `utils/biomarker_node.py:166-170`)

3.4 ParsingNeuralNetwork + GatherData (depthai-nodes glue)

`depthai-nodes 0.3.6` is the depthai_nodes Python package — a community/Luxonis helper layer above `dai.node.NeuralNetwork`. Two nodes we depend on:

- **`ParsingNeuralNetwork`** : wraps a `dai.node.NeuralNetwork` and attaches an output-specific parser. For YuNet → `ImgDetectionsExtended` parser. For Face Landmarker → `Keypoints` parser. Saves us writing manual tensor → message conversion (`main.py:104, 122`).
- **`GatherData`** : synchronizes asynchronous streams (face detections, per-face landmark messages, optionally depth) into a single `gather_data_msg` packet. We use it to bundle one detection's bbox with the landmarks it produced (`main.py:127-129`). Internally uses sequence numbers from the detection stream as the matching key.

Warning printed at startup (cosmetic, not a bug):

```
[depthai-nodes] You are using ImgDetectionsBridge to transform from
ImgDetectionsExtended to ImgDetections. This results in lose of keypoint,
```

We don't depend on the lost fields (we re-extract bbox center / size from the detection later in `BiomarkerExtractor`), so we ignore.

4. Custom Host Nodes (Mac side)

These are Python `dai.node.HostNode` subclasses — they live in our process, not on the OAK. They consume DepthAI buffers and emit further buffers.

4.1 `utils/biomarker_node.py:BiomarkerExtractor`

The main analytical node. Inputs: `gather.out` (detections+landmarks) and optionally `stereo.depth`. Output: a `dai.Buffer` carrying a JSON-encoded metrics dict (`metrics_out`).

Key state held per-instance: - `BlinkCounter` — adaptive-threshold blink detector - `AmplitudeTracker` × 5 — sliding 30 s peak-to-peak for smile/brow/asymmetry/gaze - `TremorTracker` × 2 — 10 s rolling FFT for chin and lip - Boxcar 3-frame smoothing buffer for tremor landmarks (`_chin_smooth`, `_lip_smooth`) - Camera intrinsics matrix `K` lazily fetched once size is known

Per-frame processing pipeline (`process()` method, lines 84–250): 1. Dequeue det+landmarks from gather, validate 2. Rectify landmarks: face-crop coords `[0,1]` → full-frame `[0,1]` using YuNet bbox `rotated_rect.center` ± `size/2` 3. Compute `head_yaw_deg` (cheap nose-vs-cheek-midline horizontal offset) 4. Set `off_axis = |yaw| > 25°` flag — used to gate the asymmetry/gaze updates 5. Compute and track all 6 biomarker families (see §6 for math) 6. Build the metrics JSON payload (~30 scalar fields + 108-element landmark array if recording is active) 7. Emit on `metrics_out` with preserved `timestamp` and `seqnum`

4.2 `utils/visit_writer.py:VisitWriter`

Persists data when `--no_save` is **not** set. Listens to `metrics_out`, opens 5 files at first frame:

FILE	FORMAT	SCHEMA	APPEND?
`visit_<id>.csv`	CSV (line-buffered)	22 cols: t, frame_idx, ear, ear_threshold, blink_rate, smile_, brow_, asymmetry_, gaze_, face_distance, tremor_chin/lip_*, head_yaw_deg, off_axis	append per-frame
`embed_<id>.csv`	CSV (line-buffered)	332 cols: 8 metadata + 108×3 landmark XYZ	append per-frame
`master_dataset.csv`	CSV (append-only, header on first create)	Same 332-col schema as embed_.csv	append cross-patient
`visit_<id>_frames.json`	JSON array	Mirror of visit_.csv (one object per frame, numeric types preserved)	dumped at flush
`embed_<id>.json`	JSON array	Mirror of embed_.csv	dumped at flush
`visit_<id>.json`	JSON object	Per-visit summary: mean/std/p95/min/max/trace[<=600] for each metric, plus all metadata	dumped at flush

CSV is line-buffered (`buffering=1`) so `tail -f` shows live progress. JSON mirrors are accumulated in memory and dumped atomically at flush — avoids partial-corruption if the process is killed mid-recording.

Patient registry — `data/patient_index.json` :

```
{ "next_id": 7,
  "by_name": {"demo_patient1": 1, "demo_patient_lisa": 2, "marco": 4, ...},
  "labels": {"4": 1, "6": 1}        // patient num → preset PD label
}
```

- Monotonic numeric ID assigned on first registration of each alias
- `labels` map is human-curated for known PD subjects and used to pre-fill the dashboard Record modal

4.3 `utils/annotation_node.py:OverlayNode`

Generates the live HUD on the WebRTC visualizer. Subscribes to `metrics_out` (always) and optionally to `gather.out` when `--debug` is set

- **Default mode**: prints 9 rows of formatted text on top of the camera feed (Blink, Smile, Brow, Asymmetry, Face dist, Jaw tremor, Head yaw, EAR, Elapsed). Each row colored by clinical thresholds (green/amber/red).
- **Debug mode**: also draws all 468 facial landmarks on the feed using `AnnotationHelper.draw_points`, color-coded by biomarker group: grey for the 360 unused, blue for eye points, pink for mouth, yellow for brows, green for tremor anchors (chin + lip upper), white for the asymmetry axis (nose bridge + tip).

Uses `depthai_nodes.utils.AnnotationHelper` to build `dai.ImgAnnotations` — overlay rendered by the Luxonis WebRTC visualizer client.

4.4 `utils/voice_capture.py:VoiceCaptureSession`

Host-side, **runs in parallel** to the OAK pipeline (the OAK 4 D has no mic).

- Uses `sounddevice.InputStream` to capture mic audio at 16 kHz mono float32 for `--duration` seconds in a background thread
- Writes the WAV to `/tmp` after stop
- `extract_voice_features()` is graceful: never raises, accumulates errors in an `errors` list. Lazy-imports `parselmouth`, `whisper` so missing dependencies don't break the pipeline.

Features extracted:

FEATURE	LIBRARY	REFERENCE
F0 mean / std	parselmouth.Sound.to_pitch	gold standard
Jitter (local %)	parselmouth.praat.call PointProcess	Rusz 2011 (PD jitter > 1.04 %)
Shimmer (local %)	parselmouth.praat.call	Rusz 2011 (PD shimmer > 3.81 %)
HNR (dB)	parselmouth Sound.to_harmonicity_cc	Goberman 2002 (PD HNR < 20 dB)
Intensity (dB)	parselmouth Sound.to_intensity	—
Speech rate (wpm)	openai-whisper "tiny" model **LOCAL**	Skodda 2011
Pause ratio	from Whisper segment timestamps	—
Transcript excerpt	Whisper	—

Privacy: `whisper.load_model("tiny")` runs the model **locally** on the laptop CPU (~75 MB model weights, ~3-5 s for 60 s audio). **No audio is uploaded** anywhere. The privacy story extends seamlessly to voice.

5. Backend HTTP API (Flask)

`backend/app.py`, served on port 8080. Run with `/usr/bin/python3 -m backend.app --port 8080`.

ENDPOINT	METHOD	PURPOSE
`/`	GET	Renders the dashboard template
`/api/patients`	GET	List of patient directories under `data/visits/`
`/api/patient/<id>`	GET	Aggregated patient view — series + deltas vs baseline (used by trend charts)
`/api/patient/<id>/visits`	GET	List of visits for a patient (used by the Recordings table)
`/api/patient/<id>/visit/<visit_id>`	GET	Full JSON of one visit (modal detail)
`/api/patient/<id>/visit/<visit_id>`	DELETE	Removes JSON+CSV+embed CSV+JSON+run.log
`/api/patient/<id>/log.csv`	GET	Cross-session per-patient CSV download (per-visit summary + per-frame trace)
`/api/patient/<id>/report.pdf`	GET	One-page ReportLab PDF for the patient
`/api/patient_index`	GET	Numeric-ID registry (for the Record modal label pre-fill)
`/api/record`	POST	Body: `{patient_id, duration, voice, sex, age, label, state}`. Kills debug pipeline, spawns `python3 main.py` subprocess with the right flags, watches for the visit JSON to appear, then auto-restarts the debug pipeline
`/api/recording_status`	GET	Status of the running record (active, patient, visit, csv path, last_error)
`/api/record/cancel`	POST	SIGTERM→SIGKILL the record subprocess, optionally delete partial files, restart debug pipeline
`/api/baseline`	POST	Body: `{patient_id, duration, sex, age, label, voice}`. Runs 3 sequential records (state=rest, talk, smile) with a small grace period between
`/api/baseline_status`	GET	Status of the running baseline session (current_step, completed list, last_error)

ENDPOINT	METHOD	PURPOSE
		back online (waits up to 60 s)

Process management — `_kill_existing_pipeline()` uses graceful shutdown: SIGTERM, wait up to 4 s for the process to exit (lets DepthAI tear down the DeviceGate session cleanly), then SIGKILL fallback. If we kill ungracefully, the next pipeline gets `X_LINK_DEVICE_NOT_FOUND` from the orphan gate session. The Reboot OAK button is the user-facing escape hatch when this happens.

Subprocess cleanup — we keep `proc_pid` in `_recording_state` so the cancel endpoint can target the exact process that's running, not pkill by pattern (which can race with restart).

6. Biomarker math (the heart of the value proposition)

All implemented in pure Python (numpy) in `utils/biomarkers.py`. No PyTorch, no scikit-learn. Deterministic, traceable, citable.

6.1 Blink rate — Eye Aspect Ratio (EAR)

Soukupová & Čech, "Real-Time Eye Blink Detection using Facial Landmarks", Center for Machine Perception, 2016.

```
        ||p2 − p6|| + ||p3 − p5||
EAR = ─────────────────────────────
              2 × ||p1 − p4||
```

`p1..p6` are 6 standard FaceMesh eye-contour landmarks. We use: - Left: `[33, 160, 158, 133, 144, 153]` - Right: `[263, 387, 385, 362, 373, 380]`

We average left and right EAR per frame. Open-eye baseline ≈ 0.30, blink trough ≈ 0.10–0.18. Threshold-crossing + minimum-frames-open debounce detects discrete blink events.

Adaptive threshold (`BlinkCounter.update`): - Maintains a 10 s rolling window of EAR values - After warm-up (90 frames), threshold = `v_max − 0.30 × (v_max − v_min)` — i.e. trigger when EAR drops below 70 % of the open-eye baseline - Falls back to fixed 0.22 if the dynamic range hasn't accumulated yet - **Warm-up gate**: blink_rate is `null` for the first 90 frames (~3 s) — HUD shows "Blink calibrating…", JSON has `blink_rate: null`

`rate_per_minute = (n_blinks_in_window) × 60 / window_seconds` where window grows from 1 s to 60 s as the visit proceeds.

6.2 Smile / brow amplitude in millimeters

affected first.

Steps (`utils/biomarker_node.py:155–225`): 1. Look up depth at landmark pixel using `safe_depth_at` — median in 3×3 window, NaN if all-zero 2. **Validate**: if landmark depth differs by more than 80 mm from nose-tip (anchor) depth, snap to anchor — protects against single-pixel background pixel sneaking into the median 3. **Sanity-clip** face_distance > 1500 mm → None (FaceMesh hallucinates landmarks when face leaves crop; depth then reads the back wall) 4. Back-project pixel `(u, v, d)` to world `(X, Y, Z)` mm using OAK intrinsics: `X = (u − cx) × d / fx`, `Y = (v − cy) × d / fy`, `Z = d` 5. Euclidean distance in 3D mm between landmark pairs 6. Reject if implausible: smile > 100 mm, brow > 30 mm

Specific landmark pairs: - **Smile amplitude** = `||p_61 − p_291||` (mouth corners L↔R) - **Brow amplitude** = mean of `||p_70 − p_159||` (left brow outer ↔ left eye top) and `||p_300 − p_386||` (right side)

The reported "amplitude" is **peak-to-peak** of the per-frame distance within a 30 s sliding window (`AmplitudeTracker`). When patient holds a static smile, peak-to-peak is small. When patient varies expression naturally, peak-to-peak is large. Hypomimia in PD shows up as compressed peak-to-peak across visits.

6.3 Facial asymmetry score

Djaldetti et al., "The mystery of motor asymmetry in Parkinson's disease", Lancet Neurology 2006 — PD onset is asymmetric in ~80 % of patients.

Algorithm (`asymmetry_score()`): 1. Compute the face mid-axis from nose-bridge (168) → chin (152), normalize 2. For 6 canonical L/R pairs: outer eye corners (33↔263), inner eye corners (133↔362), mouth corners (61↔291), outer brow (70↔300), inner brow (65↔295), cheek edges (234↔454) 3. Mirror each right-side point across the perp-axis of the mid-axis 4. Compute residual = mean Euclidean distance between L point and the mirrored R point 5. Normalize by `face_diag = ||p_234 − p_454||`

Result: scalar in `[0, 1+]`. Healthy ~0.02–0.05. Above 0.05 = asymmetric.

Head-pose gating: when `|head_yaw_deg| > 25°`, the asymmetry update is **skipped** (the previous valid value is retained). Without this gate, any head rotation would yield a false-positive asymmetry — at 30° yaw the mirror-residual is geometrically inflated.

6.4 Head yaw estimation

Cheap closed-form, no PnP solver (`head_yaw_deg()`):

```
yaw_deg = (nose_tip.x − midpoint(L_cheek, R_cheek).x) / |R_cheek − L_cheek| × 180°
```

Accurate to ±5° in the ±45° range — sufficient for the gating decision (threshold = 25°, so a couple of degrees of error doesn't flip the gate).

6.5 Tremor — multi-feature spectral classifier

The most complex component. Located in `TremorTracker` (`utils/biomarkers.py:189–296`).

References: - Bain, “Tremor”, Movement Disorders 2003 — limb resting tremor 4–6 Hz - Bologna 2013 — jaw/lip tremor in PD shares the same band

Per-frame inputs: - Chin position (landmark 152) and upper lip position (landmark 13) - Subtract nose bridge (168), divide by face-diagonal → **distance- and position-invariant** (a patient closer or further from the camera produces the same metric) - 3-frame **boxcar smoothing** before FFT — kills MediaPipe INT8 quantization jitter (white noise across the spectrum) without attenuating the 4–6 Hz signal (which oscillates at 5–8 frames per period at 30 FPS)

10 s ring buffer (300 samples @ 30 FPS), at every frame: 1. Detrend (subtract mean — removes DC and slow drift) 2. Hanning window (reduces spectral leakage) 3. `np.fft.rfft` → magnitude squared / `(Σ window² × fps)` → PSD 4. Compute three band powers and the dominant frequency: - `band_power_low` = sum PSD in 1–3 Hz (chewing / postural drift) - `band_power_pd` = sum PSD in 4–6 Hz (PD resting tremor band) - `band_power_high` = sum PSD in 6–12 Hz (tic / fast voluntary motion) - `motion_power` = sum PSD in 0.5–12 Hz (total jaw motion, useful as “is the patient even moving the jaw?”) - `dom_freq` = argmax of PSD in **2–12 Hz only** (skip < 2 Hz so postural drift doesn’t dominate the spectrum) 5. **Lock-in ratio** = fraction of the last 90 frames (~3 s) where dom_freq was inside 3–7 Hz. Real PD tremor sustains; chewing/talking transit.

Classifier (`pd_likelihood` , 0–100 %):

```
if motion_power < 1e-5:                    # idle, no jaw movement
    likelihood = 0
else:
    ratio = pd_power / max(low_power, eps)
    ratio_score = min(60, 30 × ln(1 + ratio))      # 0–60 points
    lock_bonus  = 40 × lock_in_ratio               # 0–40 points
    chewing_penalty = 30 if (low_power > 1.5 × pd_power
                             and motion_power > 5e-5) else 0
    likelihood = clamp(0, 100, ratio_score + lock_bonus − chewing_penalty)
```

Display in HUD: - `[–]` (green) for likelihood < 30 - `[?]` (amber) for 30 ≤ likelihood < 60 - `[PD]` (red) for likelihood ≥ 60

Tested behavior on healthy subject: - Sitting still: `0 % @ 1.5 Hz [–] (mot 5)` - Eating: `5–15 % @ 1.5–2 Hz [–] (mot 30–80)` — chewing penalty fires - Talking: `1–5 % @ 2–3 Hz [–] (mot 15)` — outside PD band - Mimicking 5 Hz jaw shake: `40–70 % @ 5.0 Hz [PD]` — passes the gate

6.6 Voice biomarkers (post-recording)

See §4.4. Computed once at end of visit, merged into `visit_<id>.json` under the `voice` key.

7. Data persistence schema

For every recording:

```
data/
├── master_dataset.csv            ← cross-patient, append-only ML dataset
├── patient_index.json            ← numeric-ID registry + PD label presets
└── visits/
    └── <patient_alias>/
        ├── visit_<id>.json       ← per-visit summary + voice features
        ├── visit_<id>.csv        ← per-frame biomarker, line-buffered
        ├── visit_<id>_frames.json ← JSON mirror of the CSV above
        ├── embed_<id>.csv        ← per-frame ML embedding (332 cols)
        ├── embed_<id>.json       ← JSON mirror of the embed CSV
        └── visit_<id>.run.log    ← stdout/stderr of the pipeline subprocess
```

`embed_<id>.csv` schema (the **important one** for ML):

COLUMN BLOCK	COLS	DETAIL
metadata	9	`patient_num, patient_id, visit_id, timestamp_iso, timestamp_ms, sex, age, label, state`
landmarks	324	`x_<i>, y_<i>, z_<i>` for each `i` in `LOWER_FACE_FIXED_INDICES` (108 indices)
total	333	one row per frame (~25 rows per second)

Curated lower-face landmark whitelist (`LOWER_FACE_FIXED_INDICES` , `utils/biomarkers.py:62-94`): 108 MediaPipe FaceMesh indices covering lips (outer + inner contour), nose, chin, jaw line, lower cheeks. **Eyes and brows are deliberately excluded** — clinically motivated (Bologna 2013: PD facial masking dominates the lower face, upper face stays expressive longer). This matches our hypothesis slide.

Coordinates are full-frame normalized `[0, 1]` (post-rectification through the YuNet bbox). `z` is MediaPipe's relative depth (unitless head-pose proxy, not metric — depth in mm is captured separately in the biomarker CSV under `face_distance_mm`).

Timestamp: - `timestamp_ms` = Unix epoch milliseconds (sortable, deterministic) - `timestamp_iso` = ISO 8601 with milliseconds (`2026-05-10T01:32:46.234` , human-readable)

`visit_id` is now used purely as a **session group key** (for `groupby` in pandas), not as a frame identifier — that's the timestamp's job.

8. Frontend (dashboard at :8080)

Single Jinja template `backend/templates/dashboard.html`. Stack: vanilla JavaScript + Plotly.js 2.35.2 + JetBrains Mono / Inter (Google Fonts).

Visual identity: deep navy `#0A1230` near-black with sparse cyan `#22DDFF` accents. Subtle 40 px grid background pattern. Mono font for metadata and labels. Style aligned with the pitch deck.

Components: - **Header**: SVG logo (3 circles cyan/blue/dark-blue on navy), wordmark “NeuroVista · Parkinson Screening”, patient dropdown, action buttons (Refresh, Download CSV, Download PDF, Record visit, Run baseline, Reboot OAK) - **Summary KPI grid**: 8 cards (4 facial, 4 voice) with delta vs. baseline - **Recordings table**: list of all visits for the selected patient, click-row-for-detail, Delete button per row - **Trend charts**: Plotly line+marker, 6 facial + 6 voice, on shared visit-id x-axis, with green-tinted “normal range” rectangle overlays - **Modals**: - Record visit (8 fields) - Run baseline (wizard with 3 stages: rest → talk → smile, each ~10 s) - Recording detail (full JSON breakdown of one visit) - **Sticky banners**: Record progress with cancel button; reboot status

JS asynchronous flow for Record: 1. `POST /api/record` returns immediately with `started: true` 2. JS polls `GET /api/recording_status` every 1 s 3. Banner shows progress “Recording — Xs/Ys” 4. When `s.active === false`, banner becomes success/error and patient list auto-refreshes (in case directory changed)

9. Validation summary (real recordings)

Tested on 8 healthy subjects (hackathon participants). Highlights:

STATE	FRAMES	BLINK/MIN	SMILE MM	ASYM	TREMOR PD%	LOCK-IN	OFF-AXIS%
rest (calm baseline)	187–353	28–31	33–36	0.022–0.027	0.0	1 %	0–0.6 %
smile (held expression)	183–359	36–57	22–49	0.04–0.05	0.0	0 %	7–13 %
talk (rainbow passage)	188–353	53–74	17–35	0.04–0.17	1–4 %	30–45 %	12–100 %
visit (60 s free)	1054–1071	45–79	45–61	0.06–0.13	0 %	4 %	28–70 %

Headlines: - **PD likelihood always < 5 %** for healthy subjects, even during talking with high spectral lock-in — the scoring penalty correctly suppresses voluntary motion. - **Smile peak-to-peak amplitude**

% gradient - **Brow amplitude** rises from 6.8 mm (rest) to 10.5 mm (smile) — 50 % increase, consistent with the orbicularis oculi activation - **Head-pose gate** keeps asymmetry clean during free movement: when off-axis fraction climbs to 28 %, asymmetry stays gated rather than drifting

These numbers will form Slide 5 of the pitch deck if you want to show “early traction” data.

10. Known limitations / V2 roadmap

LIMIT	CAUSE	FIX PATH
FPS effective ≈ 18 vs. 30 target	RVC4 quantized YuNet+FaceMesh+stereo combined latency	Quantize to lower bitwidth, reduce stereo resolution, profile bottleneck (likely the Script crop block)
TremorTracker uses fixed `fps=30` while real ≈18	Hardcoded constructor argument	Measure FPS dynamically from `t` deltas (last 30 frames) and pass it. Currently the dominant_hz reading is scaled — multiply by ratio for true Hz
YuNet bbox loses confidence > 30° yaw	Single face-detector limitation	Fallback face detector (RetinaFace as second pass) or 3DDFA full mesh which is robust to large yaw
Iris not tracked	Luxonis Zoo only ships 468-pt FaceMesh, not 478 (with iris)	Convert MediaPipe Face Landmarker v2 (478) using `luxonis/modelconverter` (~2-3 h)
EAR threshold sensitive to glasses	Fixed `0.22` floor + adaptive but starts conservative	Adapt to per-patient baseline taken from `state=rest` recording
Personal noise floor not subtracted	Baseline subtraction not yet wired	Read `data/visits/<patient>/visit_*_rest.json`, average tremor `motion_power` and `band_power_pd`, subtract from current visit before classification
Voice depends on Mac mic + TCC permission	Foundational — OAK 4 D has no mic	Move voice processing to a dedicated USB mic on the OAK PoE+ board if RVC4 ever exposes ALSA
Device crashes after ~10–15	Likely thermal on RVC4 DSP	Active heatsink / fan, or cycle the device between back-to-back recordings (already done in our

11. Why this stack and not X

Predictable Q&A:

Q: Why OAK 4 D and not a regular webcam + cloud? A: (1) Privacy is the product — patient video never leaves the device. (2) Stereo depth is load-bearing — smile amplitude in mm, not pixels, is clinically meaningful. Pixel deltas aren't (they confound with distance). (3) On-device runtime cost = electricity. Cloud cost = per-API-call, unbounded scaling.

Q: Why MediaPipe and not OpenFace 2.0? A: OpenFace exposes 17 Action Units (FACS-grade), which would be the most clinically rigorous choice for hypomimia (AU6 cheek raiser, AU12 lip corner puller, etc.). But OpenFace is not in the Luxonis Zoo and converting it to RVC4 is a 1-2 day effort with no guarantee. MediaPipe Face Landmarker is in the Zoo, INT8 RVC4 ready, and 468 landmarks are sufficient to compute EAR + smile + brow + asymmetry + tremor proxies. AU support is on the V2 roadmap.

Q: Why Whisper-tiny locally and not OpenAI API? A: API would round-trip patient audio to OpenAI servers — incompatible with the privacy story. Whisper-tiny runs in 3-5 s on Mac CPU for 60 s of audio, no network needed. The accuracy difference for our needs (word count for speech rate, segment timestamps for pause ratio) is negligible — we don't need high-accuracy transcription.

Q: Why 6 biomarkers and not just the most validated one (jitter)? A: Single-biomarker screening is unreliable. Each metric individually has high false-positive rate at the population level (jitter 1.04 % cutoff catches 30 % of healthy subjects too). The clinical insight is that **multimodal multi-biomarker ensemble** is what discriminates — and this needs to be built once for all biomarkers.

Q: Why fixed `LOWER_FACE_FIXED_INDICES` whitelist and not the dynamic `y > eye_line_y` check? A: The whitelist gives **stable CSV columns** across visits (essential for ML training where features must align row-to-row). Dynamic mask would yield variable column count per frame. Whitelist is curated manually from the canonical FaceMesh schema, ~108 points covering all PD-relevant lower-face regions.

12. Quickstart for a fresh Mac

```
# 1. Python 3.9 + voice deps (host-side)
/usr/bin/python3 -m pip install --user \
    sounddevice soundfile praat-parselmouth openai-whisper

# 2. Pre-download Whisper-tiny model (avoids first-run latency in demo)
/usr/bin/python3 -c "import whisper; whisper.load_model('tiny')"

# 3. Confirm depthai stack is on Python 3.9
/usr/bin/python3 -c "import depthai, depthai_nodes, cv2, numpy; \
    print(depthai.__version__, depthai_nodes.__version__)"
# Expected: 3.6.1 0.3.6

# 4. Boot OAK over PoE+, verify
oakctl device list
# Expected: 1 device, IP 169.254.x.x, OS RVC4 ≥ 1.32.0

# 5. Smoke test (no save, no voice, debug landmarks)
cd ~/Downloads/GDG-AI-Hack-2026/neurosight
/usr/bin/python3 main.py --no_save --no_voice --debug

# 6. Open visualizer
open http://localhost:8082

# 7. Start dashboard backend in a second terminal
/usr/bin/python3 -m backend.app --port 8080
open http://localhost:8080
```

If the device crashes or recordings start failing with `X_LINK_DEVICE_NOT_FOUND`, click "↻ Reboot OAK" in the dashboard header (or `oakctl device reboot -d <ip>` from terminal).

13. Academic references (cite these in Q&A)

1. **Karson CN.** Spontaneous eye-blink rates and dopaminergic systems. Brain 1983;106:643-653 → blink rate < 12/min as PD biomarker
2. **Soukupová T, Čech J.** Real-Time Eye Blink Detection using Facial Landmarks. 21st Computer Vision Winter Workshop 2016 → EAR formula

4. **Djaldetti R et al.** The mystery of motor asymmetry in Parkinson's disease. Lancet Neurol 2006;5(9):796-802 → asymmetric onset
5. **Bain PG.** Tremor. Mov Disord 2003;18(suppl 8):S5-S15 → 4–6 Hz resting tremor band
6. **Pretegiani E, Optican LM.** Eye Movements in Parkinson's Disease and Inherited Parkinsonian Syndromes. Front Neurol 2017;8:592 → saccade instability
7. **Rusz J et al.** Quantitative acoustic measurements for characterization of speech and voice disorders in early untreated Parkinson's disease. J Acoust Soc Am 2011;129:350-367 → jitter, shimmer thresholds
8. **Goberman AM, Coelho C.** Acoustic analysis of Parkinsonian speech I. NeuroRehabilitation 2002;17:237-246 → HNR reduction
9. **Skodda S et al.** Progression of dysprosody in Parkinson's disease over time — A longitudinal study. Mov Disord 2009;24:716-722 → speech rate slowing

Disclaimer to repeat in Q&A: **NeuroVista is a research / screening prototype, not a medical device.** All clinical decisions require a licensed neurologist. Reference ranges from cited literature are indicative, not diagnostic.